

New Open SQL

D75AW

Figure 85: SQL Expressions - The Need

Unit 5 Lesson 2

Before:

```
SELECT carrid connid fldate seatsmax seatsocc
FROM sflight INTO gs_flight.

gs_flight-seatsfree = gs_flight-seatsmax
                    - gs_flight-seatsocc.

APPEND gs_flight TO gt_flights.

ENDSELECT.
```

With NW 7.40:

```
SELECT carrid,
       connid,
       fldate,
       seatsmax,
       seatsocc,
       seatsmax - seatsocc AS seatsfree
FROM sflight
INTO TABLE @gt_flights.
```

New Open SQL

Operators

- +, -, *, /

Numeric Functions (NW 7.40 SP05)

- ABS, CEIL, FLOOR, DIV, MOD

Numeric Functions (NW 7.50)

- ROUND

Brackets

- "(" and ")" To influence the priority of expressions
- Spaces needed after "(" and before ")"

```
SELECT carrid,  
       seatsmax,  
       seatsocc,  
       seatsmax - seatsocc AS seatsfree,  
       seatsocc * 100 / seatsmax AS occupancy_rate  
FROM sflight  
INTO TABLE @DATA(lt_flights).
```

```
SELECT CEIL( price ) AS price_up,  
       FLOOR( price ) AS price_down  
FROM zproducts  
INTO TABLE @DATA(lt_prod).  
SELECT amount DIV 100 AS hundreds,  
       amount MOD 100 AS remainder  
FROM ztable  
INTO TABLE @DATA(lt_data).
```

```
SELECT ROUND( price, 2 ) AS rounded_price // 소수점 처리  
FROM zproducts  
INTO TABLE @DATA(lt_prod).
```

```
SELECT ( seatsmax - seatsocc ) * 100 / seatsmax AS rate  
FROM sflight  
INTO TABLE @DATA(lt_data).
```

New Open SQL

D75AW

Figure 88: Example: CAST-Expression for Numeric Types

Unit 5 Lesson 2

Before:

```
SELECT carrid connid fldate seatsmax seatsocc  
FROM sflight INTO gs_flight1.
```

```
gs_flight-percentage =  
    gs_flight1-seatsocc / gs_flight1-seatsmax * 100.  
APPEND gs_flight TO gt_flights.  
ENDSELECT.
```

이러면 정수 나눗셈이라 소수점이 잘림

ex) $75 / 100 = 0 \rightarrow 0 * 100 = 0$

With NW 7.40:

```
SELECT carrid, connid, fldate, seatsmax, seatsocc,  
    CAST( seatsocc AS FLTP ) /  
    CAST( seatsmax AS FLTP ) *  
    CAST( 100 AS FLTP ) AS percentage  
FROM sflight  
INTO TABLE @gt_flights2.
```

(정수끼리 나누면
소수점이 버려짐)

나눗셈 연산자(/)는
부동소수점 계산에서만
제대로 작동합니다

그래서 계산에 사용되는
모든 값들을 FLTP(부동소수점)
타입으로 변환

단, 타입 캐스팅은 무조건 FLTP로 하는 것이 아니라,
계산 목적(통화, 수량, 비율 등)에 따라 적절한 타입을 선택하는 것이 더 중요하다.
사실 모든 값을 FLTP로 할 필요는 없고, 나눗셈 연산할 때 피연산자/연산자 중
둘 중 하나만 골라서 FLTP로 캐스팅해도 잘 작동한다.

New Open SQL

D75AW

Figure 89: Example: String-Expression

Unit 5 Lesson 2

Before:

```
SELECT id name form
FROM scustom
INTO gs_customer1
gs_customer1-full_name = gs_customer1-form && ' ' &&
                        gs_customer1-name.
APPEND gs_customer1 TO gt_customers1.
ENDSELECT.
```

With NW 7.40:

```
SELECT id,
       name,
       form,
       form && ' ' && name AS full_name
FROM scustom
INTO TABLE @gt_customers2.
```

New Open SQL

Simple Case Distinction:

```
SELECT carrid, connid, fldate, bookid,  
CASE class  
  WHEN 'Y' THEN 'Economy'  
  WHEN 'C' THEN 'Business'  
  WHEN 'F' THEN 'First'  
END AS class_txt  
FROM sbook INTO TABLE @gt_bookings
```

Complex Case Distinction :

```
SELECT carrid, connid, fldate,  
CASE  
  WHEN seatsocc * 100 < seatsmax * 50 THEN @icon_green_light  
  WHEN seatsocc * 100 < seatsmax * 90 THEN @icon_yellow_light  
  ELSE @icon_red_light  
END AS icon_occ  
FROM sflight INTO TABLE @gt_flights2.
```

New Open SQL

1. 인터널 테이블은 빈 테이블인지 체크 필수 → **비면 내용 전부 담음**
2. 연결되는 인터널 테이블의 필드를 기준으로 정렬을 꼭 해주기 → 속도업
3. 정렬된 Fields를 기준으로 중복값을 제거
4. 데이터의 손실이 발생할 우려가 있으므로 별도의 백업 ITAB을 이용하여 중복값 제거 및 DB 테이블과 연결

Before:

```
SELECT FROM spfli FIELDS carrid, connid
      WHERE cityfrom = 'FRANKFURT' INTO TABLE @gt_connections.

SELECT FROM sbook FIELDS ...
      FOR ALL ENTRIES IN @gt_connections
      WHERE carrid = @gt_connections-carrid
      AND connid = @gt_con
      INTO ...
```

WITH 절 (CTE - Common Table Expression)

+connections라는 임시 결과 집합을 with 절로 만들 (+ 기호 주목!)
이 결과는 마치 인터널 테이블처럼 사용할 수 있음

With NW 7.51

```
WITH +connections AS ( SELECT FROM spfli
                        FIELDS carrid, connid
                        WHERE cityfrom = 'FRANKFURT' )

SELECT FROM +connections INNER JOIN sbook
      ON +connections~carrid = sbook~carrid
      AND +connections~connid = sbook~connid
      FIELDS ...
```

New Open SQL

Before:

```
SELECT agencynum name street city country  
FROM stravelag  
INTO TABLE gt_partners.
```

```
SELECT id name street city country  
FROM scustom  
APPENDING TABLE gt_partners.
```

Two individual
statements,
into same target

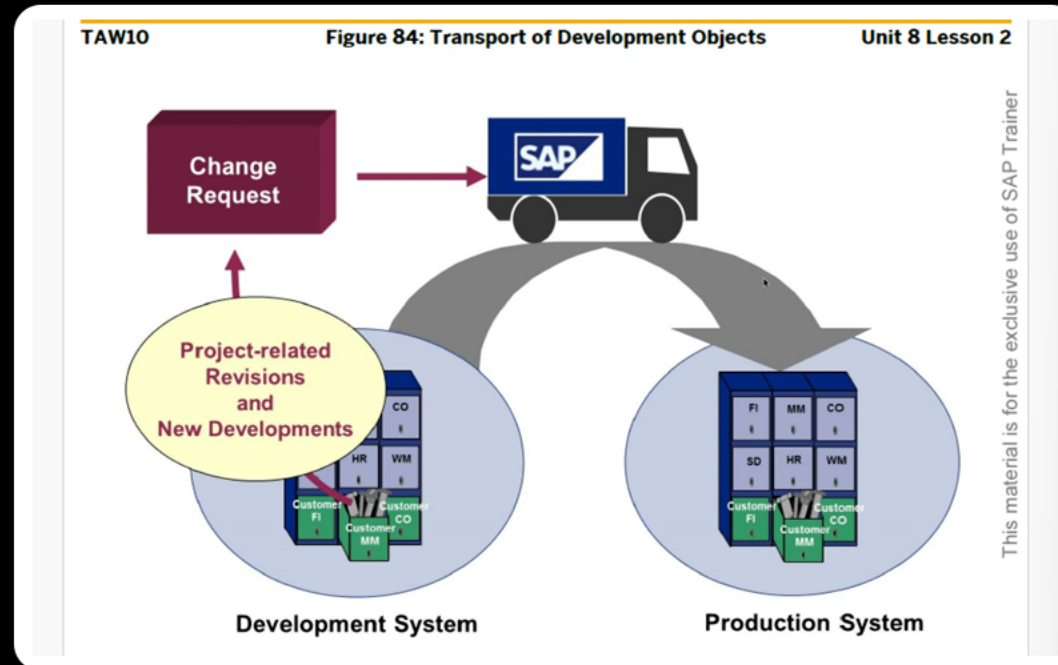
With NW 7.50

```
SELECT agencynum, name, street, city, country  
FROM stravelag  
UNION  
SELECT id, name, street, city, country  
FROM scustom  
INTO TABLE @gt_partners2.
```

One statement,
combination done
on database

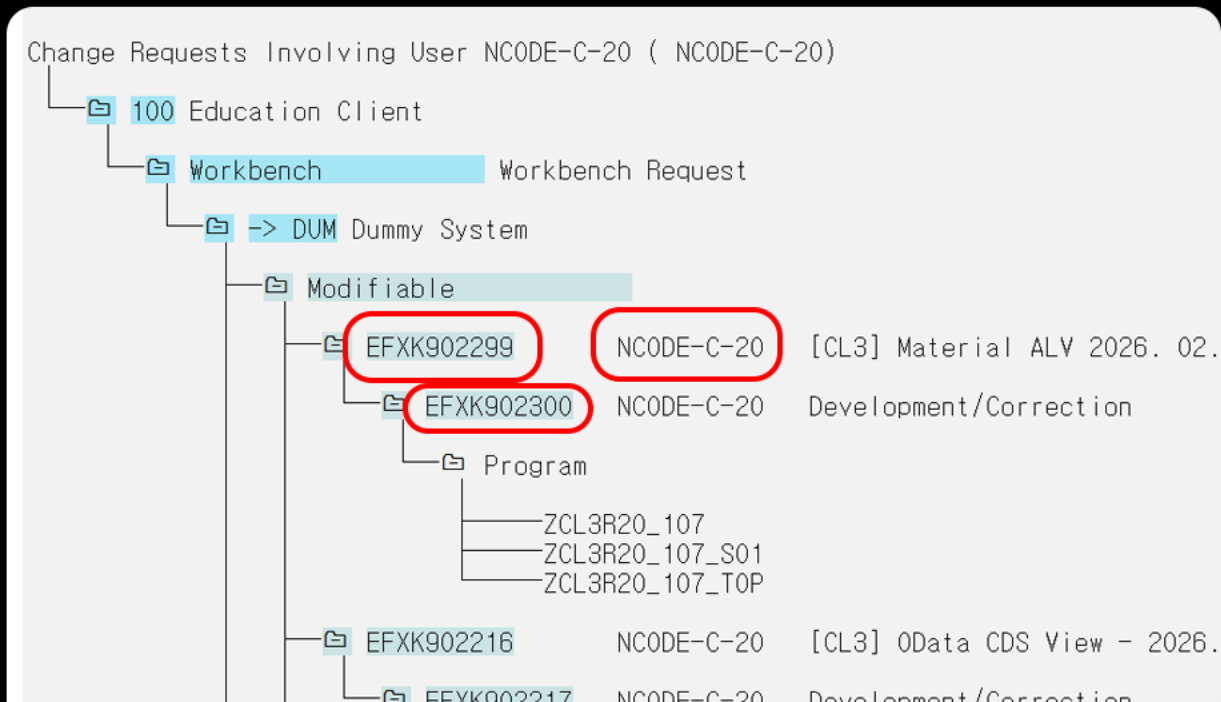
Change and Transport System (CTS)

- SAP 시스템 간에 **개발 객체, 커스터마이징 설정, 그리고 기타 변경사항을 이동시키는 도구**
 - 일반적으로 개발 서버 (DEV) → 품질 서버 (QA) → 운영 (PRD) 시스템으로 **변경사항을 전달할 때 사용됨**
- **모든 프로그램을 한 CTS 번호에 몰아넣는건 매우 위험하다.**
 - 프로그램 단위로 CTS 번호를 따로 만드는데 좋고, 연관된 것들(진짜 말 그대로)이 있다면 꼭 같이 넣어야한다.
 - 통으로 만들면 QA에서 일부 프로그램만 테스트하는 것이 불가능해지므로, 나누는 것이 좋다.



Change and Transport System (CTS)

- 한 CTS 번호에는 하위에 Task 번호가 계속 추가되는 방식이다.
 - CTS 번호는 소유자가 있으며, 그 CTS 번호에 다른 사용자 번호를 추가할 수 있다.
 - CTS에 사용자 번호를 추가하면 → Task Number가 하나 더 추가되는 방식이다.
 - 이렇게 하면 여러 명이 작업한 것들을 안전하게 이관할 수 있음
 - Task Number에는 관리를 용이하게 하기 위해서 어떤 내용이 들어있는지 Description을 잘 써야만 한다.



Association in CDS View

- CDS에서 다른 View(또는 테이블)와의 관계를 정의해두는 "지연(Deferred) LEFT OUTER JOIN"
 - JOIN을 바로 수행하는 것이 아니라 관계만 정의해두고 실제로 사용할 때 JOIN이 발생
 - Where 조건 사용 불가 / ON 절에는 \$projection을 꼭 써야한다
- 그래서 이거 왜 쓰는데? → 단순 SQL이 아닌 데이터의 정의서인 CDS View에서 테이블 간의 관계를 정의하려고!
- 쓰면 뭐가 좋은데? → Exposed로 해두면 재사용성이 좋음, 필요할 때만 조인하면 되므로 성능 좋음

S4D430

Figure 120: On-Condition with \$Projection

Unit 4 Lesson 2

```
@AbapCatalog.sqlViewName: 'S4D430_ASS03'
```

```
define view S4D430_Association_3 as select
  from spfli as c
    association[1..1] to scarr as _Carrier

    on $projection.CarrierID = _Carrier.carrid ,
  {
    key c.carrid as CarrierID,
    key c.connid,
    c.cityfrom,
    c.cityto,
    _Carrier.carrname
  }
```

Use \$projection on the left hand side of the ON-clause

If the field has an alias, the alias has to be used after \$projection

Cardinality in Association

- 카디널리티 표기는 선택사항이고, 기본 카디널리티 설정은 `[0..1]`이다.
- 최소값은 기본값이 `0` 이므로, 생략가능하다.
 - 최소값이 1이 아니라 0인 이유는, 연결된 레코드가 없을 수도 있기 때문
 - 이걸 LEFT OUTER JOIN이므로 0으로 해야하는게 맞다, [1...*]으로 하면 INNER JOIN이 되어버린다
 - 그래서 최소값을 생략 가능한 것
- [1] 은 [0..1], [4] 는 [0..4]
 - [*] 는 [0..*], *는 여러 개(n)의 레코드가 있을 수 있다는 말이다.
- 최솟값과 최댓값에는 입력금지되는 값이 있다 → 사실 당연한 소리긴 함
 - 최솟값은 `\*` 입력금지
 - 최댓값은 `0` 입력금지
- Association은 Where 조건을 사용할 수 없다.
 - 이걸 왜일까?
 - LEFT OUTER JOIN에 WHERE 걸지말라고 했잖아

S4D430

Figure 1

```
@AbapCatalog.sqlViewName
define view S4d430_Association
from spfli as c
  association[*] to sfli
  on $pr
  and $pr

{
  key carrid,
  key connid,
```

Open SQL에서 Association 사용하기

```
SELECT carrid, connid,  
       \_Carrier-carrname,           " Association 사용!  
       c~\_Carrier-currcode          " Alias와 함께 사용  
FROM s4d430_association_exposed AS c  
WHERE c~\_Carrier-currcode = 'USD'  
INTO TABLE @gt_data.
```

Association Filter Condition

S4D430

Figure 131: Example: Filter Condition in Path Expression

Unit 4 Lesson 2

```
@AbapCatalog.sqlViewName: ,S4D430_ASSOFILT'
define view S4d430_Association_FILTERED
  with parameters
    language : syst_langu @<Environment.systemField: #SYSTEM_LANGUAGE

  as select
    from scarr as a
      association to tcurt as _Currency
      on $projection.currcode = _Currency.waers
  //
    and _currency.spras = $parameters.language
  {
    carrid,
    carrname,
    currcode,
    $parameters.language as lang,

    _Currency[spras = $parameters.language].ktext as currency_name
  }
```

ON 절에 언어 조건을 넣으면 하나의 언어로만 고정됨
다른 언어로 사용하고 싶을 때 Association을 재사용할 수 없어진다
Association의 유연성이 사라짐

사용(호출) 시점에 원하는 언어를 선택할 수 있어서 유연하게 사용 가능
같은 CDS View를 여러 언어로 재사용 가능
파라미터로 동적 제어가 가능해짐

```
SELECT FROM S4D430_Association_FILTERED(
  language = 'KO' // 한국어
)
```

Cardinality Changing in Filter Condition

S4D430

Figure 133: Example: Cardinality in Filtered Associations

Unit 4 Lesson 2

```
@AbapCatalog.sqlViewName: 'S4D430_FILTCARD'
@EndUserText.label: 'Demo: Cardinality in Filtered Asso.'
define view S4D430_Asso_Filter_Cardinality
  with parameters
    language : syst_langu @<Environment.system
as select
  from scarr as a
    association[*] to tcurt as _Currency
    on $projection.currcode = _Currency.waers
  {
    carrid,
    carrname,
    currcode,
    $parameters.language as lang,
    _Currency[1:spras = $parameters.language] as _Currency_name
  }
```

tcurt는 통화 텍스트 테이블 → 언어별 데이터가 있음
원래 카디널리티는 [0..*] 즉 1:n 관계에 놓여있음
하나의 통화코드에 따라 여러 텍스트를 가질 수 있단 의미를 정의

그런데 여기서 필터가 들어감
파라미터로 특정 언어(spras)만 선택하여
"이 통화 금액에서 현재 시스템 언어에 해당하는 텍스트만 가져와라"
그래서 특정 언어인 1건만 가져오게 되므로 카디널리티가 [0..1]이 됨
1:1 관계가 되어버린다는 뜻